

Model Engineering

Virtsudo

May 17, 2026

1 Frequency-Based Baseline

In this section we built *Frequency-based dataframes*. We took this approach as baseline, next we trained and tested simple Machine Learning model using them.

1.1 Data Analysis

Initially, there were two datasets train and test, each was collection of samples, where sample contains `api_call_sequence` (list of API calls) and `is_malware` labels (1 is malware, and 0 is normal). Both train and test datasets were split to sequences and labels, and we built vocabulary, a sorted set of API calls. On the train set, there are **258** distinct API calls, while on the test set **232**. Discovered three API calls (`'ControlService'`, `'NtDeleteKey'`, `'WSASocketA'`), which were appeared only on test set. They were assigned with special token `<UNK>`.

A feature set was created from the train vocabulary plus the `<UNK>` token, where each feature is assigned a unique integer index based on its position in the sorted vocabulary. A frequency vector was built for each sample by counting the occurrences of each API call at its corresponding *index* position, and collected for each dataset. As previously mentioned, API calls that do not exist in the train vocabulary are counted under the `<UNK>` token.

A sparsity analysis was performed, since it can directly affect the *frequency-based* approach. The average number of non-zero elements per row in the train set was **21.95**, with a ratio of only **0.08** with respect to the total number of features. On the test set, the average was **24.28** with a ratio of **0.09**. While the *frequency-based dataframes* serve as a good baseline, they lose the original API call order, since they focus only on occurrence counting.

1.2 Classifier and Results

Logistic Regression was selected as a basic Machine Learning model, and used `GridSearchCV` with $C \in \{0.01, 0.1, 1.0, 10.0\}$, 3-fold CV, scoring on F1, which helps to find best fit hyperparameters. For better handle unbalance data, `class_weight` set to 'balanced'. The best C was selected automatically by `GridSearchCV`, final test performance achieved **0.64** macro *F1-score* and **0.81** *Recall* for *Normal* class, which is reasonably strong for a simple frequency baseline. Despite **92%** sparsity and losing API order, the macro *F1-score* stayed **0.64**, which proved *frequency-based* approach alone carried real signal for malware detection.

2 FeedForward Neural Network

In this section we focused on implementation of *FeedForward Neural Networks*. In more detail, datasets were preprocessed especially for FFNN models, then two models were trained and tested.

Preprocessing. Compared to *frequency-based* approach, FFNN requires a *fixed-size* input vectors. Therefore, we first analyzed the sequence length distribution across both datasets. As shown in Fig. 1, train set sequences are distributed between **60** and **90** with a median of **75.0**, while the test set has higher lengths between **70** and **100** with a median of **87.0**. Since FFNN works only with *fixed-size* input vectors, a global `max_len=90` parameter was chosen as the maximum sequence length from the train set, avoiding data leakage from the test set. To normalize all sequences to this fixed size, *padding* and *truncation* were applied. A special `<PAD>` token was introduced, assigned an index following the `<UNK>` token in the feature set. Each sequence shorter than `max_len` was padded with `<PAD>` tokens,

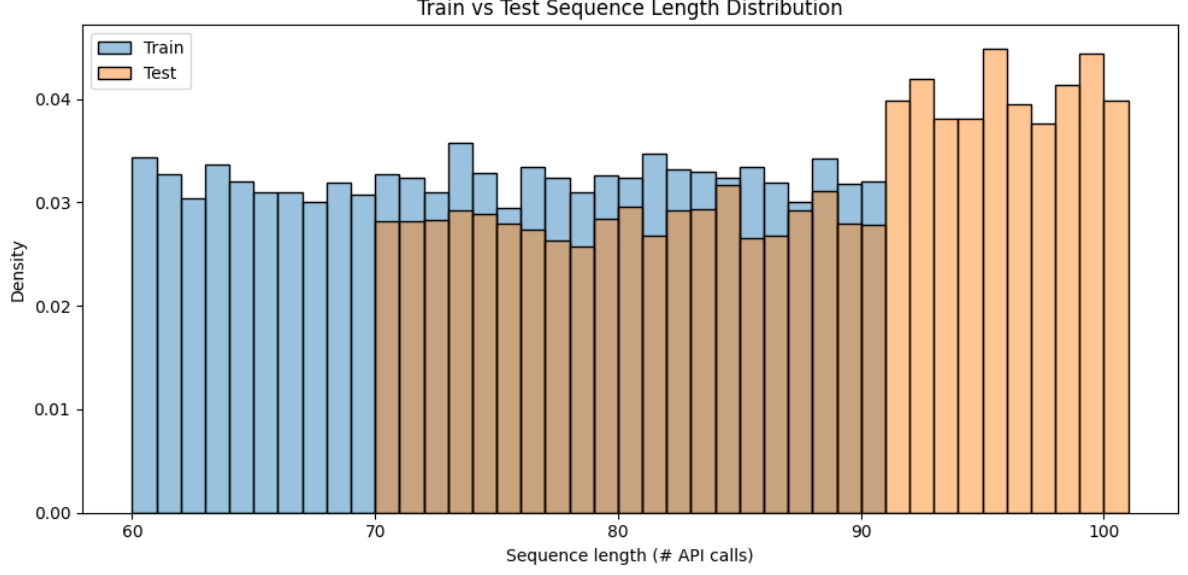


Figure 1: Density histogram of sequence lengths for train and test sets, showing that test sequences tend to be longer than train sequences

while sequences exceeding `max_len` were truncated to the first `max_len` elements. The final preprocessing step was converting each API call sequence into a sequence of integer indices. Two encoding approaches were explored: *Sequential Identifiers*, where each API call is mapped to a fixed integer index based on its position in the sorted vocabulary, and *Learnable Embeddings*, where each API call is mapped to a trainable dense vector of size `embed_dim`.

Training and Testing.

Some hyperparameters were shared across both FFNN models, while others were tuned individually based on model performance. Specifically, a batch size of **64** and **100** epochs with early stopping were used for both. As the loss function, `BCEWithLogitsLoss` was selected. Since the dataset is heavily imbalanced toward the *Malware* class, `pos_weight` was set accordingly to balance the class contributions during training. For both models, the `AdamW` optimizer was used.

The first FFNN model, based on *Sequential Identifiers*, used the following hyperparameters: input dimension set to `max_len`, hidden dimension of **64**, and output dimension of **1** as required by the loss function. The learning rate was set to **1e-4** to avoid overshooting given the sparse gradient signal, and dropout was not applied, since the model already struggles to learn meaningful representations from raw integer indices.

The second FFNN model, based on *Learnable Embeddings*, used the following hyperparameters: vocabulary size equal to the length of the feature set, embedding dimension of **64**, learning rate of **1e-3** since embeddings provide richer gradient signal, and dropout of **0.2** to prevent overfitting.

Finally, both models were trained and tested. While both converged during training, the *Sequential Identifier* model failed significantly on the *Normal* class, achieving a macro *F1-score* of only **0.43**. As shown in Fig. 2, the model misclassified **2148** (33%) *Malware* samples as *Normal*, which is critical from

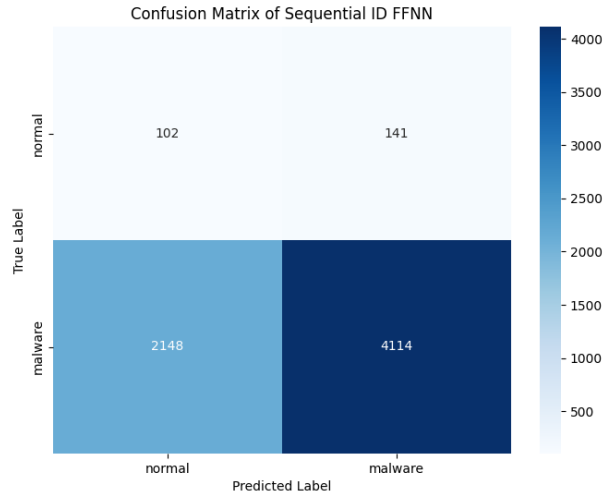


Figure 2: Confusion matrix of Sequential ID model, showing model misclassification on *Normal* class

a Cybersecurity perspective. This collapse is explained by the nature of integer IDs — they carry no semantic meaning, so the model cannot learn useful patterns from them and struggles to distinguish between classes.

The *Learnable Embedding* model performed significantly better than the *Sequential Identifier* model, achieving a macro *F1-score* of **0.63**. By learning dense vector representations, the model captured more meaningful patterns than relying solely on raw integer indices. However, as shown in Table 1, the baseline still slightly outperformed the *Learnable Embedding* FFNN. This suggests that even with richer input representations, FFNN still ignores the sequential order of API calls, which limits its ability to capture behavioral patterns beyond what simple frequency counting already provides.

Classification Report						
	Baseline			FFNN		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Normal	0.22	0.81	0.35	0.21	0.73	0.32
Malware	0.99	0.89	0.94	0.99	0.89	0.94
accuracy			0.89			0.89
macro avg	0.61	0.85	0.64	0.60	0.81	0.63
weighted avg	0.96	0.89	0.91	0.96	0.89	0.92

Table 1: Baseline and *Learnable Embeddings* FFNN model class report, showing the baseline performed better than FFNN model.

3 Recurrent Neural Network

In this section we turned to more complex Deep Neural Network models like *Recurrent Neural Networks*, the datasets were preprocessed, as RNN models requires different preprocessing procedures than FFNN. Next, three RNN models were trained and tested: *One-directional RNN*, *Bidirectional RNN* and *LSTM* models.

Preprocessing.

Compared to FFNN preprocessing, RNNs are more memory efficient, as they use *dynamic padding* based on the *per-batch maximum length* rather than a global one. This avoids unnecessary padding, since the local batch maximum is typically smaller than the global maximum. Another advantage of RNNs over FFNNs is that truncation is not required — variable-length sequences are handled via `pack_padded_sequence`, which processes only real tokens and avoids losing useful data. Sequences were sorted by length within each batch for efficient packing.

Training and Testing. The hyperparameters for RNN models selected as follows: the same batch size as FFNN (64), lower learning rate 1e-4, 100 epochs with early stopping (min delta=1e-4, patience=8). The vocabulary size taken as length of `api_to_idx` map (near the same as feature set). Embedding dimension equal to 64, while hidden dimension is **256** with two layers, so that the models better fit on data. We kept the same loss function (`BCEWithLogitsLoss`) with `pos.weight` parameter on all models to better compare them, and output dimension is 1 respectively. As regularization *dropout* used with value **0.3**.

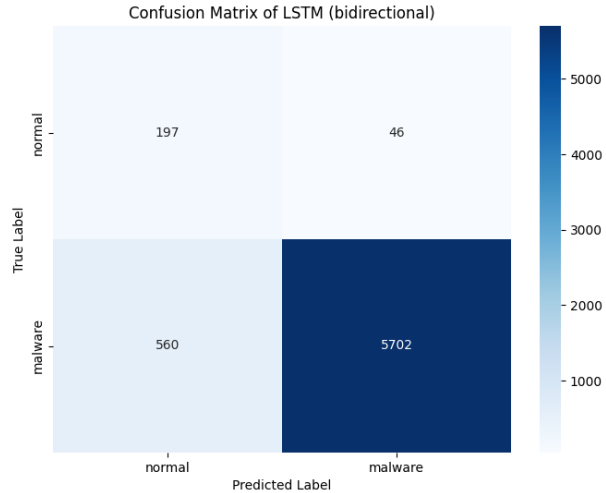


Figure 3: Confusion matrix of Bi-LSTM model, showing the model well classified

Three models were trained and tested. Compared to FFNN, RNN models required significantly more training time, as each output depends on the previous one through the `hidden_state`, forcing sequential processing — unlike FFNN which can be fully parallelized.

The RNN models showed less stable performance comparing to FFNN models, because of backpropagation through time over long sequences (90 steps), RNN model suffered from vanishing(or exploding) gradients problems. Even with smaller learning rate, first two models still had lower macro *F1-score* than baseline model, both *One-directional RNN* and *Bidirectional RNN* had macro *F1-score* equal to **0.61**.

The *Bi-LSTM* model (Fig. 3) was the only model to surpass the baseline, achieving a macro *F1-score* of **0.67**. Its *gating mechanism* mitigates vanishing gradient problems and better captures long-range dependencies in API call sequences. This confirms that architectural complexity alone is insufficient, the right mechanism must be chosen to handle the specific properties of the data.

Finally, a comparison between the *Baseline* and *Bi-LSTM* models is presented in Table 2, showing that the Bi-LSTM model outperformed the baseline not only in macro *F1-score*, but across all reported metrics.

	Classification Report					
	Baseline			Bi-LSTM		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Normal	0.22	0.81	0.35	0.26	0.81	0.39
Malware	0.99	0.89	0.94	0.99	0.91	0.95
accuracy			0.89			0.91
macro avg	0.61	0.85	0.64	0.63	0.86	0.67
weighted avg	0.96	0.89	0.91	0.96	0.91	0.93

Table 2: Baseline and *Bi-LSTM* models class report, showing the Bi-LSTM model performed better than baseline model

4 Graph Neural Network

In this section we preprocessed the data, by converting API call sequences to graph nodes, and edges. Then the three GNN models were trained and tested (GCN, GraphSAGE, GAT).

Preprocessing. Unlike FFNN and RNN models, GNNs do not require *padding* or *truncation*, as they naturally handle *variable-sized* graph structures. Each unique API call was represented as a node, and a directed edge was added between two nodes if one API call was immediately followed by another in the sequence. Self-loops were allowed. Edges are unweighted, meaning only the relationship between API calls is captured, the frequency of transitions and the exact order of calls are lost.

Training and Testing. The hyperparameters for GNN models selected as follows: the same batch size as for previous models (64), learning rate of 1e-4, 100 epochs with early stopping (min delta=1e-3, patience=8). As regularization `weight_decay` (1e-4) used instead of dropout, the loss function (BCEWithLogitsLoss with `pos_weight`) the same as before for all models, and output dimension is 1 respectively. The number of nodes is equal to number of unique features, embedding and hidden dimensions are 64, and only for *GAT* model heads parameter used with value of 4, since it used multiple attention heads.

All three GNN models converged during training, however only *GraphSAGE* reached a macro *F1-score* equal to the baseline. *GCN* achieved **0.62**, falling below the baseline, as it treats all neighbors equally, losing important structural details. *GAT* also achieved **0.62** — despite its powerful attention mechanism, node features consisted only of integer indices, providing insufficient signal for meaningful attention weights, leading to underperformance. Finally, *GraphSAGE* performed best among GNN models with a macro *F1-score* of **0.64**, tying the baseline but unable to surpass it. As shown in Fig. 4, while *GraphSAGE* achieved higher metrics on the *Malware* class, the *Normal* class recall of the baseline

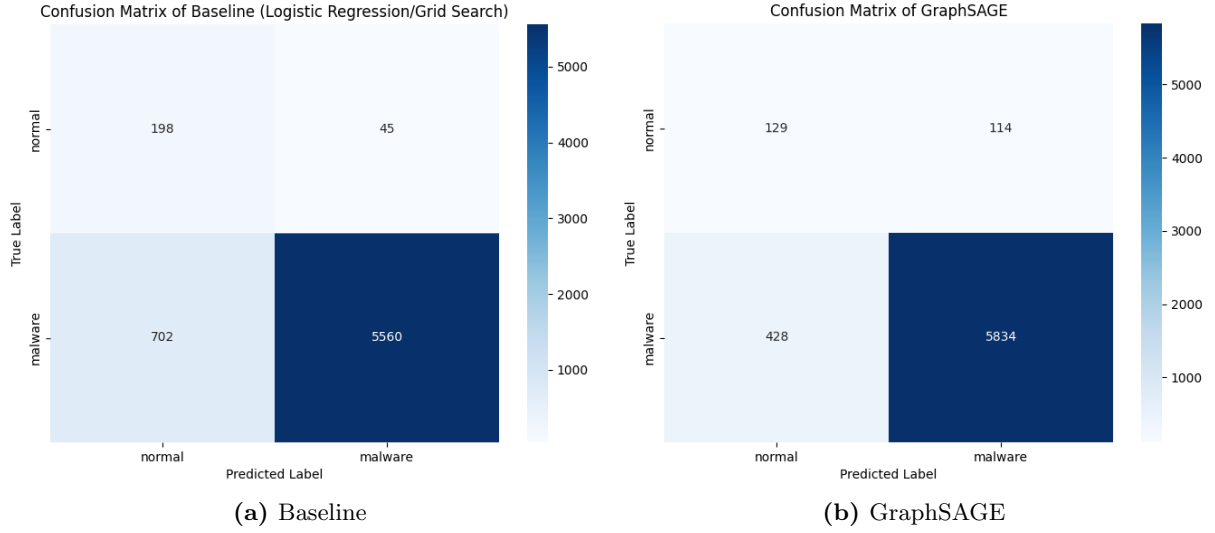


Figure 4: Baseline and GraphSAGE confusion matrices, showing that despite equal macro $F1$ -score, GraphSAGE misclassifies significantly more *Normal* samples

(**0.81**) was significantly higher than that of *GraphSAGE* (**0.53**). This is critical from a cybersecurity perspective, as misclassifying normal samples as malware leads to false alarms.

5 Overall Comparison

As shown in Fig. 5, the macro $F1$ -score and *Normal* class *Recall* are compared across the baseline and all Neural Network models. The best performing model was *Bi-LSTM* with a macro $F1$ -score of **0.67**, while also matching the baseline in *Normal* class *Recall* (**0.81**). The second best model was the simple *frequency-based* baseline, which proves that sometimes simple approaches can outperform more complex solutions. The worst performing model was the *Sequential Identifier* FFNN, which relied on raw integer indices without learning meaningful relationships or useful patterns.

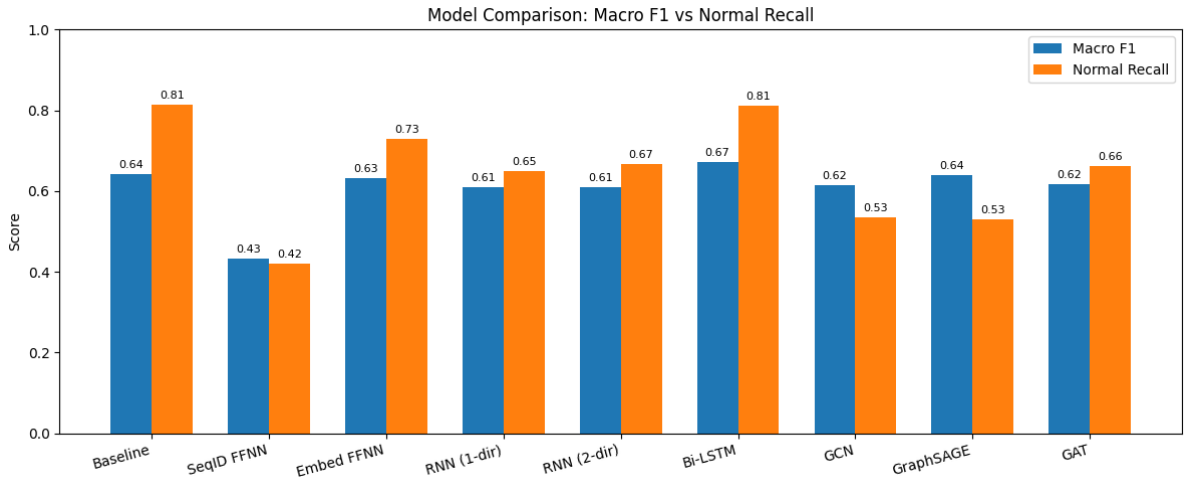


Figure 5: The baseline and all Neural Network models compared with macro $F1$ -score and *Normal* class *Recall*